

Recommandation de Films à partir des Données TMDB et Netflix

Un Moteur de Recherche Personnalisé



SOMMAIRE

1. Introduction	3
2. Analyse descriptive et visualisations des données	3
2.1 Description générale et exploration des données	3
2.2 Analyse graphique des notes moyennes par genre	5
2.3 Répartition des années de sortie des films sur TMDb et de leur ajout sur Netflix	5
2.4 Analyse des critères déterminants pour la note et la popularité des films	7
3. Méthodologie	10
3.1 Les étapes de développements	10
3.1.1 EXPLORATION DES JEUX DE DONNÉES	10
3.1.2 CONCATENATION	11
3.1.3 MODIFICATIONS POUR NOS UTILISATIONS FUTURES	11
3.1.4 ANALYSE DESCRIPTIVE ET VISUALISATION DES DONNÉES	12
3.1.5 MOTEUR DE RECHERCHE/RECOMMANDATIONS	12
3.1.6 CRÉATION D'INTERFACE GRAPHIQUE	12
3.1.7 MISE AU PROPRE	13
3.2 Les outils utilisés	13
3.3 Les bibliothèques utilisées :	14
3.4 Répartition des tâches	14
4. Conception	15
4.1 Architecture du programme	15
4.2 Interface graphique	16
5. Implémentation	17
5.1 Fonctionnalités développées	17
5.2 Exemple de code pertinent	17
5.2.1 FILTRER LES FILMS SELON DES CRITERES DE PREFERENCE	17
5.2.2. EXEMPLE : EXPLICATION DANS L'INTERFACE DE L'AFFICHAGE DU BOUTON2	18
5.2.3 EXEMPLE : EXPLICATION DANS L'INTERFACE DU TRAITEMENT DU BOUTON2	19
5.2.4. EXEMPLE : EXPLICATION D'AFFICHAGE DE 2 DATA FRAMES SUR L'INTERFACE	19
6. Tests et validations	20
7. Perspectives	21
7.1 Les améliorations possibles	21
7.2 Apprentissages et difficultés rencontrées au cours du projet	22
8. Annexes	23
8.1 Les instructions pour lancer notre programme	23
8.2 Bibliographie	24

1. Introduction

Comment trouver des films qui correspondent parfaitement à vos goûts tout en découvrant de nouveaux films ?

C'est à cette question que nous avons souhaité répondre avec notre application : **NetFilm**.

Intuitive, dynamique et innovante, cette application a pour objectif d'aider les amateurs de cinéma à découvrir de nouveaux films parfaitement adaptés à leurs goûts. Grâce à un moteur de recherche performant, NetFilm analyse les critères ou les films que l'utilisateur apprécie pour lui proposer des suggestions sur mesure.

Vous n'avez pas d'idée de film ? Rejoignez NetFilm et laissez-nous vous proposer une vaste sélection de films, adaptée à vos préférences.

2. Analyse descriptive et visualisations des données

2.1 Description générale et exploration des données

Vous devrez tout d'abord effectuer une description classique des données (ex. nombre d'observations, de variables, type des variables...) puis répondre aux questions suivantes :

Pour approfondir la description et la compréhension de nos données, nous avons examiné les statistiques descriptives globales afin de dégager une vue d'ensemble de leurs principales caractéristiques.

Nos données regroupées sous forme d'un unique DataFrame contiennent 1 125 325 films. Le jeu de donnée contient 33 variables.

Les variables telles que 'id', 'vote_count', 'vote', 'revenue', 'runtime' et 'budget' sont de type 'int64' soient des entiers.

Les variables telles que 'vote_average' et 'popularity' sont de type 'float64', soient sous forme de chiffres décimaux.

La variable 'adult' est une variable booléenne de valeur True ou False.

Le reste des variables ont le type 'object', soit le type utilisé par défaut par pandas.

Nom de variable	Type de donnée	Nombre de valeurs NaN
id	int64	0
title	object	0
vote_average	float64	0
vote_count	int64	0
status	object	0
release_date	datetime64[ns]	182083
revenue	int64	0
runtime	int64	0
adult	bool	0
backdrop_path	object	825690
budget	int64	0
homepage	object	1005940
imdb_id	object	525610
original_language	object	0
original_title	object	0
overview	object	226645
popularity	float64	0
poster_path	object	352566
tagline	object	967401
genres	object	0
production_companies	object	616404
production_countries	object	0
spoken_languages	object	0
keywords	object	0
year_release	object	182083
key	object	182083
director	object	1120370
cast	object	1120618
country	object	1120533
date_added	object	1120254
rating	object	1120256
listed_in	object	1120254
description	object	1120254

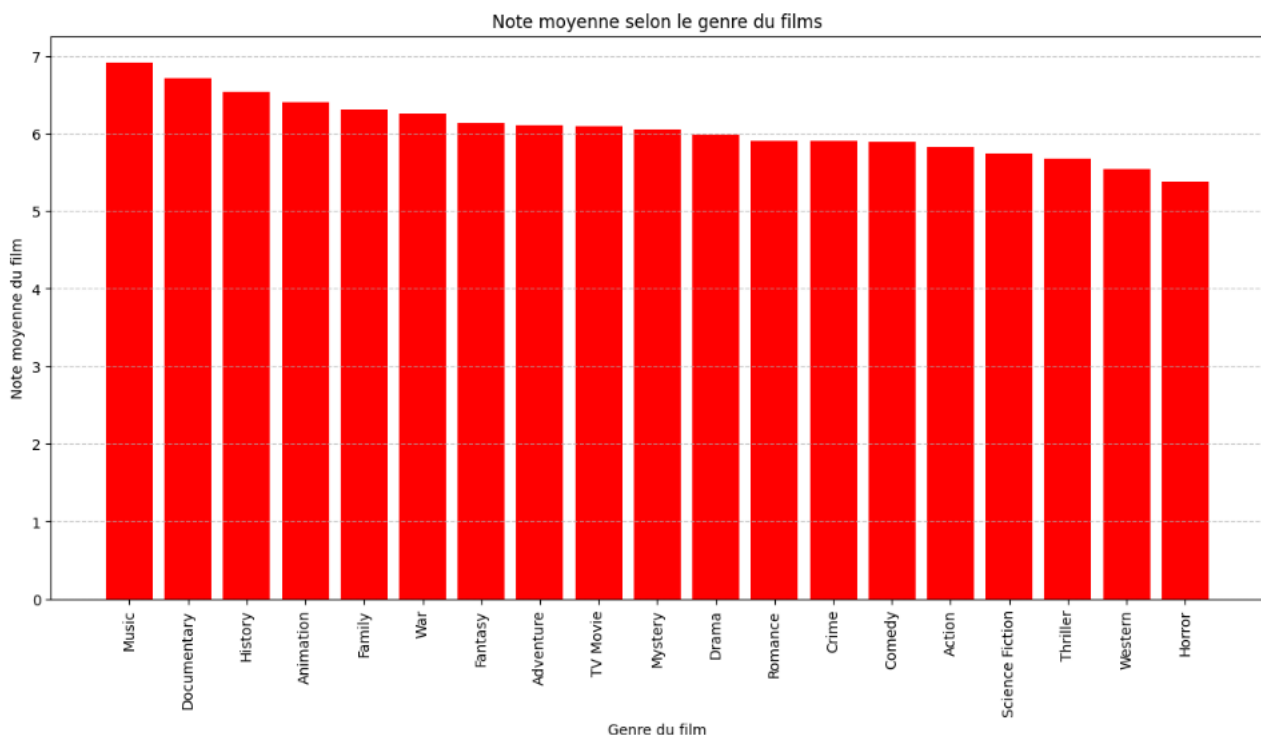
On peut noter que certaines variables contenues dans notre DataFrame global qui proviennent du data set de Netflix contiennent un nombre de NaN égal au nombre de films contenus dans le dataset TMDB.

	id	vote_average	vote_count	release_date	revenue	runtime	budget
count	1125325.00	1125325.00	1125325.00	943242	1.125325e+06	1125325.00	1.125325e+06
mean	734126.29	1.90	19.06	2000-02-24	6.943787e+05	47.94	2.692397e+05
min	2.00	0.00	0.00	1800-01-01	1.200000e+01	-28.00	0.000000e+00
25%	407571.00	0.00	0.00	28/06/1989	0.000000e+00	0.00	0.000000e+00
50%	738716.00	0.00	0.00	25/11/2010	0.000000e+00	23.00	0.000000e+00
75%	1081891.00	4.80	1.00	01/01/2019	0.000000e+00	89.00	0.000000e+00

Concernant la description classique des données, certaines données sont faussées, par exemple le film le plus court dure -28min. La moitié de nos données ont des notes égales à 0, ce que l'on peut interpréter comme une absence de notes pour près de la moitié des films, la note maximum vaut 10. Le film le plus vieux est sorti en 1800 et les films de cette base de données datent de l'année 2000. Le film le plus long, dure 24h. Les films ont été réalisés avec en moyenne un budget de 1 125 325 \$.

2.2 Analyse graphique des notes moyennes par genre

Nous avons analysé les notes moyennes par genre de film afin de mieux comprendre les préférences des individus et d'identifier quels sont les genres les plus populaires. Pour cela, nous avons pensé à remplacer les valeurs dont les notes sont égales à 0 par la médiane globale de tous les genres. Cependant au vu des résultats obtenus nous avons choisi de ne pas tenir compte des médianes dans nos données puisque cela aurait faussé nos moyennes (réhaussant certains genres selon le nombre de vote = 0).

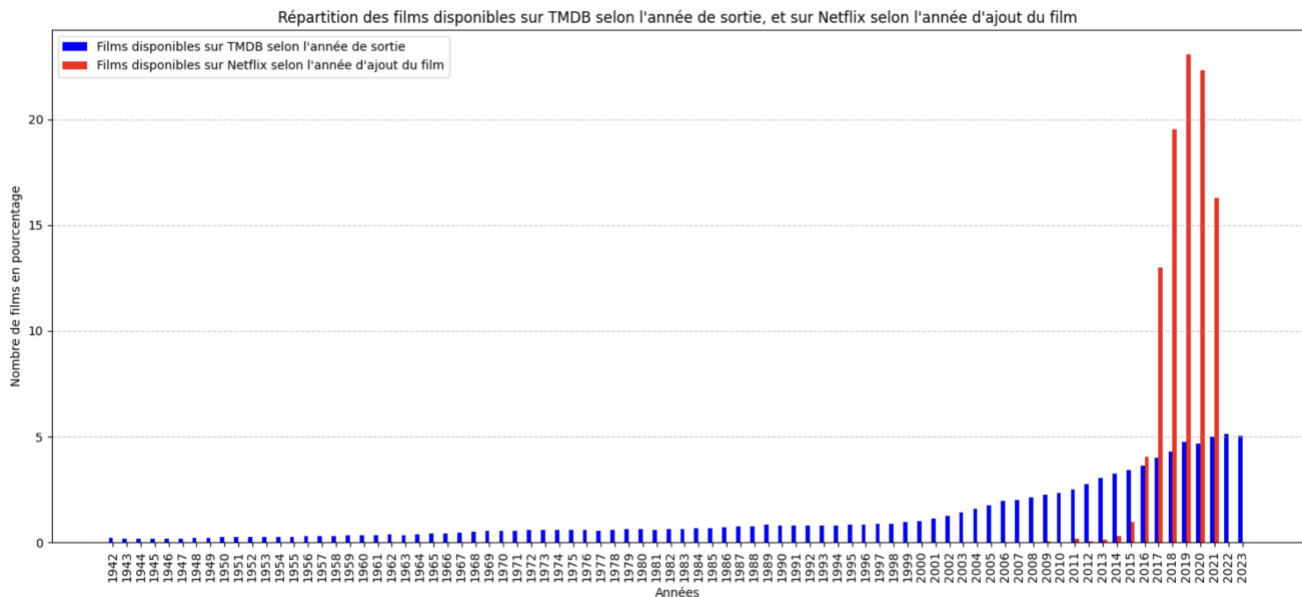


D'après le graphique obtenu, le genre le plus populaire concerne les films de type 'Music' et le deuxième genre le plus populaire est le genre 'documentaire' ce qui nous paraît étonnant. Néanmoins, on peut remarquer que notre analyse a été réalisée après avoir retiré les valeurs NaN ou les valeurs égales à 0, et que les notes ne concernent qu'un certain nombre de films. Le genre de film le plus populaire est le genre 'Horror'. On peut également dire que la note moyenne du genre de film le plus populaire est d'environ 6,8 tandis que la note des films d'horreur (genre le moins bien noté) est de 5,5.

2.3 Répartition des années de sortie des films sur TMDB et de leur ajout sur Netflix

Nous avons également trouvé pertinent d'analyser la répartition des dates de sorties de films présents sur TMDB par rapport à ceux ajoutés sur Netflix. Pour ce faire, nous avons choisi de commencer l'analyse à partir de 1942 puisque c'est l'année de sortie du film le plus ancien disponible sur Netflix. En effet, inclure les films datant d'avant 1942 aurait introduit des données qui ne sont pas comparables, puisqu'aucun film de cette période n'est accessible sur Netflix. Ce choix garantit une analyse ciblée et cohérente en se concentrant uniquement sur la période où les deux bases, Netflix et TMDB, peuvent être mises en relation de manière pertinente.

Nous obtenons donc ce graphique :



A noter que le nombre de films est affiché en pourcentage (du nombre de films disponibles selon l'année de sortie ou de la date d'ajout du film), ce qui permet une meilleure visualisation puisqu'autrement les films Netflix ne représentaient qu'une infime partie de nos données totales et n'étaient pas bien visibles.

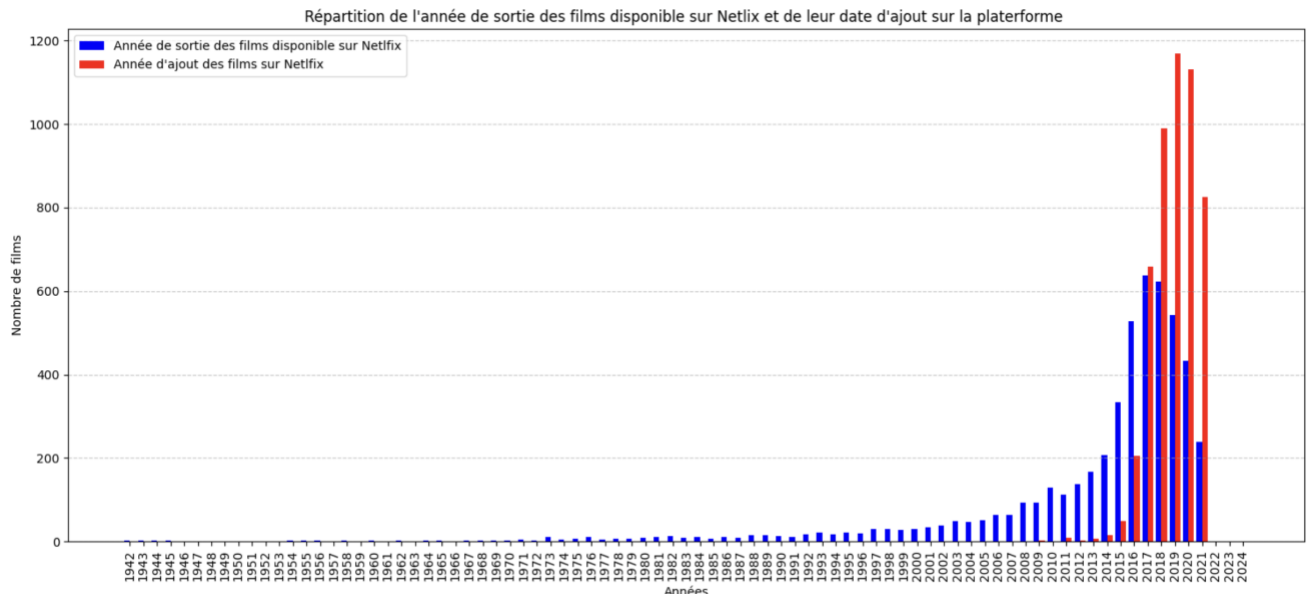
Concernant l'analyse graphique, on observe dans un premier temps une évolution constante de la production de films depuis 1942, avec un nombre croissant de films chaque année. Toutefois, si nous examinons de plus près, on remarque qu'entre 1942 et 2002, la progression est très faible, suivie d'une période de stagnation. Ce n'est qu'à partir de 2004 environ que nous constatons une augmentation significative du nombre de films. De plus, TMDB présente un large éventail de films provenant de différentes périodes, bien que la proportion de films récents (à partir de 2000) soit plus élevée.

En ce qui concerne les films ajoutés sur Netflix, le service de vidéo à la demande par abonnement a débuté en 2007. Cela explique la faible proportion de films ajoutés durant les premières années. Cependant, à partir de 2015, cette proportion augmente considérablement. Netflix a ajouté une grande partie de ses films entre 2017 et 2021, avec une forte hausse en 2019 et 2020, probablement en raison de la pandémie de COVID-19, durant laquelle la demande a considérablement augmenté. Netflix a su répondre à cette demande en ajoutant massivement de nouveaux films.

Ici, nous avons conservé tous les films de TMDB dans un premier DataFrame, tandis que dans un second, nous avons filtré uniquement les films disponibles sur Netflix. Cela signifie qu'il existe des films présents dans TMDB qui ne sont pas forcément disponibles sur Netflix. C'est pourquoi nous avons décidé de créer un graphique distinct montrant les années de réalisation des films TMDB qui

sont exclusivement disponibles sur Netflix. Nous avons ensuite comparé ces données avec leurs dates d'ajout sur Netflix, ce qui nous a permis d'identifier les périodes de production que Netflix privilégie le plus dans l'enrichissement de sa base de données.

Nous avons obtenu ce graphique :



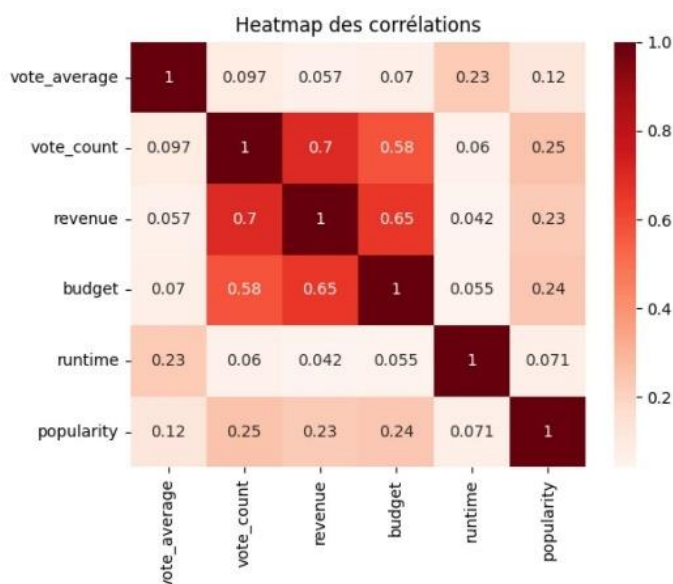
Ce graphique est presque identique au précédent, surtout en ce qui concerne les dates d'ajout des films sur Netflix. Cependant, il permet d'observer plus clairement la répartition des années de sortie des films au cinéma, ce qui nous aide à identifier les années les plus présentes sur Netflix. On remarque ainsi une faible proportion de films sortis avant 2009, ce qui indique que Netflix privilégie l'ajout de films plus récents sur sa plateforme. De plus, une certaine latence apparaît entre la date de sortie des films et leur ajout sur la plateforme. Bien que des analyses plus approfondies puissent être faites pour mieux comprendre ce phénomène, cela n'est pas l'objectif ici. Néanmoins, il est raisonnable de penser que Netflix n'ajoute pas immédiatement un film après sa sortie en salle, car il faut du temps pour acquérir les droits de diffusion et négocier les contrats nécessaires.

2.4 Analyse des critères déterminants pour la note et la popularité des films

Analyse des facteurs influant sur la note : Analyse des facteurs influant sur la popularité : Quels critères influent sur la note d'un film ? Utiliser des graphiques pour illustrer ces relations, et mettre en avant les tendances importantes identifiées

Nous avons ici cherché à analyser et expliquer les relations des caractéristiques de films avec deux de nos variables : 'vote_average' et 'popularity'.

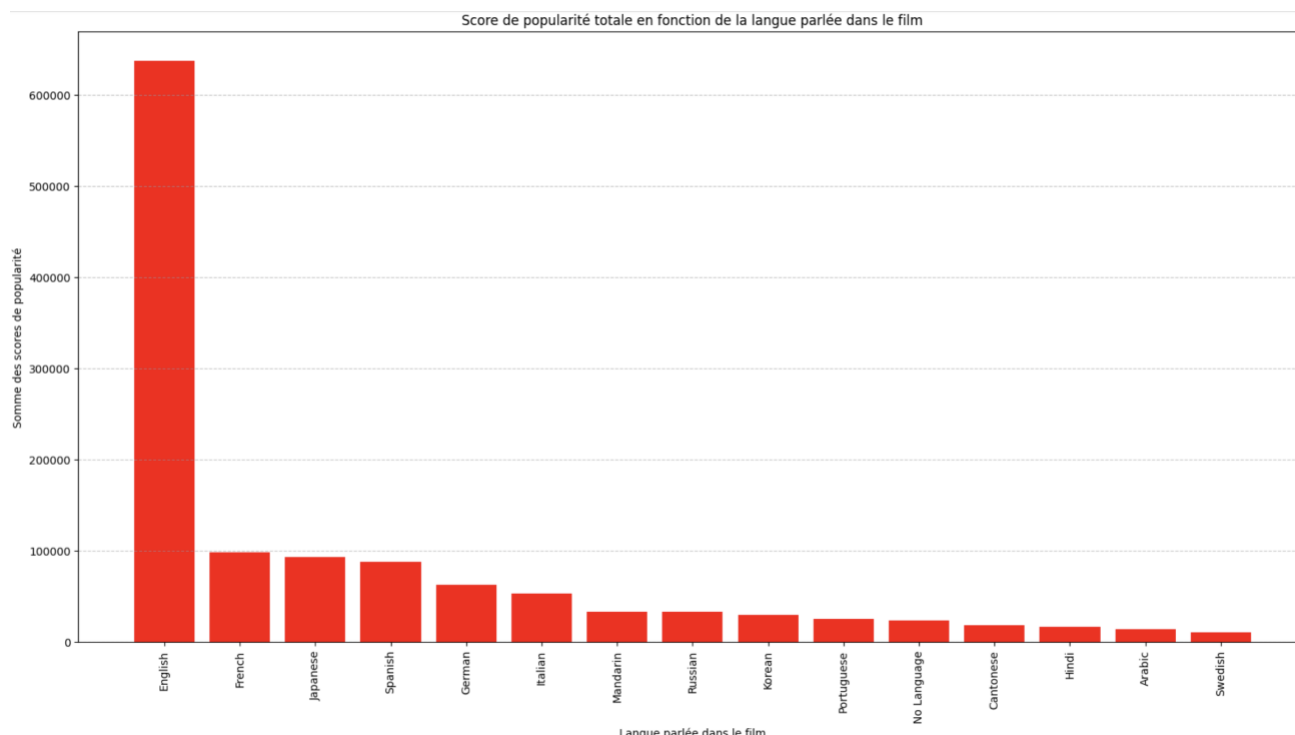
Afin de déterminer quelles étaient les caractéristiques les plus pertinentes à analyser pour démontrer une certaine relation entre la note d'un film/ sa popularité et le reste de nos variables nous avons tout d'abord pensé à calculer la corrélation de notre Dataframe entre toutes les variables. Nous avons donc obtenu ce heatmap avec comme représentation visuelle :



D'après les résultats des corrélations entre les variables, on constate que la popularité d'un film est expliquée à 24 % par le budget consacré, à 25 % par le nombre de votes reçus, et à 22 % par les revenus générés. En revanche, la note moyenne d'un film n'est globalement pas influencée par les autres variables de notre DataFrame. Il est toutefois intéressant de noter que la durée et la popularité d'un film expliquent respectivement 22 % et 11 % de la note moyenne, bien que ces corrélations soient faibles et insuffisantes pour tirer des conclusions solides.

L'analyse du heatmap révèle des corrélations significatives entre certaines variables, par exemple une corrélation de 70 % entre les revenus d'un film et le nombre de votes reçus. Cependant, aucune variable quantitative ne semble jouer un rôle déterminant sur la popularité ou la note moyenne d'un film.

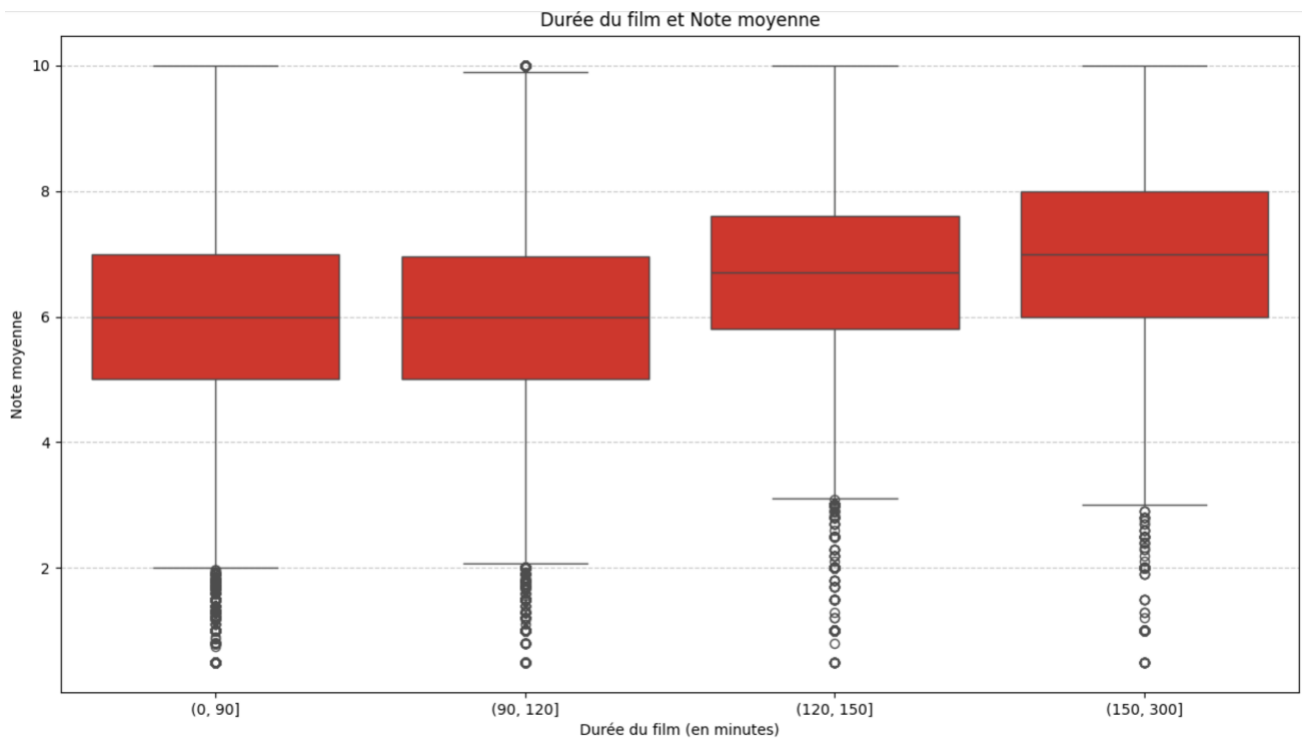
En conséquence, nous avons décidé de nous concentrer sur les variables qualitatives. Parmi celles analysées, la durée du film et la langue parlée sont apparues comme les plus influentes sur la note attribuée à un film.



Dans notre analyse des variables qualitatives, nous avons choisi de représenter les 15 langues les plus parlées dans les films en fonction de la somme totale des scores de popularité des films. Cette méthode a été privilégiée car, en utilisant la moyenne, les langues très peu représentées auraient affiché des moyennes de popularité artificiellement élevées, ce qui aurait biaisé l'interprétation des résultats.

Les résultats montrent que les films en anglais atteignent un score de popularité total de plus de 600 000, soit plus de six fois celui des films en français, qui occupent la deuxième position avec un score de 100 000. Cette différence souligne l'écart considérable entre les films en anglais et ceux dans d'autres langues en termes de popularité totale.

On observe également que les films en français, japonais, espagnol, allemand et italien, bien qu'à une grande distance de l'anglais, présentent des scores de popularité totaux relativement similaires. En revanche, pour les langues suivantes, telles que le mandarin ou le russe, une diminution notable du score de popularité est visible.



Les médianes des notes de films sont très proches pour les trois premières catégories de durées (0-90 minutes, 90-120 minutes, et 120-150 minutes), avec une valeur de 6/10. Cependant, on observe une légère augmentation de la médiane pour les films plus longs, allant de 150 à 300 minutes, ce qui suggère que les films les plus longs tendent à être mieux notés.

En ce qui concerne la dispersion des données, les trois premières catégories présentent une dispersion similaire, avec des écarts interquartiles homogènes : le premier quartile (Q1) se situe autour de 5/10 et le troisième quartile (Q3) autour de 7/10. Pour la dernière catégorie (150-300 minutes), les écarts interquartiles sont plus resserrés, indiquant une plus grande cohérence dans les notes attribuées aux films les plus longs. On note également la présence de nombreuses valeurs aberrantes, principalement des films ayant des notes inférieures à environ 2/10, situées en dessous du premier quartile.

Enfin, il est possible de supposer que les films les plus longs obtiennent de meilleures notes en raison d'un développement narratif plus approfondi et complet.

3. Méthodologie

3.1 Les étapes de développements

3.1.1 EXPLORATION DES JEUX DE DONNEES

Avant de commencer les tâches demandées, il a été nécessaire de prendre connaissance des jeux de données mis à disposition. Nous avons donc commencé par les importer, puis nous avons analysé les colonnes afin de comprendre leur signification, leur type et leur rôle. Certains premiers ajustements ont été effectués, comme le changement de formats pour certaines colonnes ou la création de nouvelles colonnes pour préparer les futures analyses.

En explorant les données, nous avons remarqué une incohérence dans le jeu de données des films Netflix : la colonne « rating » contenait des valeurs qui semblaient correspondre à des durées de films. Nous avons corrigé cette erreur en transférant ces valeurs dans la colonne appropriée, qui était vide pour ces lignes. D'autres incohérences similaires ont été identifiées, mais nous avons choisi de ne pas les traiter systématiquement, car ce n'était pas l'objectif principal du projet. Par ailleurs, il est normal qu'un jeu de données ne soit pas totalement exempt d'erreurs.

3.1.2 CONCATENATION

Une fois les jeux de données analysés et compris, une étape cruciale a été leur concaténation. Cette tâche s'est révélée particulièrement complexe, car il n'existait pas de clé de concaténation déjà disponible dans les jeux de données. Il a donc fallu en créer une. Après plusieurs essais, nous avons décidé de construire une clé basée sur les noms des films et leurs années de réalisation.

Cependant, en utilisant uniquement les données brutes présentes dans les jeux de données, la qualité de la concaténation était insuffisante. Par exemple, avec cette méthode, nous ne pouvions récupérer qu'environ 1000 entrées du jeu de données Netflix. Face à ce problème, nous avons pensé à utiliser la distance de Levenshtein pour reconnaître les films dont les titres étaient presque identiques entre les deux bases. Au final, cela n'était pas vraiment pertinent. C'est pourquoi, nous avons développé une fonction pour normaliser les titres : tous les titres ont été convertis en minuscules, les symboles spéciaux tels que «:;,'_-&» ont été supprimés, et les espaces répétitifs également. Cette normalisation visait à atténuer les divergences liées à des différences mineures de syntaxe, notamment pour les titres de sagas par exemple.

Une fois cette clé de concaténation finalisée, nous avons procédé à une jointure gauche. Cela nous a permis de conserver l'intégralité des données du jeu TMDB, tout en ajoutant les informations du jeu Netflix lorsque des correspondances étaient trouvées via la clé. Grâce à cette méthode, nous avons réussi à atteindre un taux de récupération de 80 % des données Netflix.

Nous avons ensuite vérifié la présence éventuelle de doublons parmi ces 80 %. Environ 300 doublons ont été identifiés, principalement dus à la présence, dans la base TMDB, de films partageant le même titre et la même année de réalisation. Ces doublons ont été conservés, bien que cela entraîne l'inclusion d'informations sur des films qui ne figurent pas réellement sur Netflix. Nous avons pris note de cette limite et en sommes conscients.

3.1.3 MODIFICATIONS POUR NOS UTILISATIONS FUTURES

Après ces étapes, nous avons apporté des modifications pour faciliter les traitements ultérieurs. Par exemple, dans la colonne des genres, nous avons transformé les chaînes de caractères en listes en utilisant la fonction « split », car les genres étaient initialement séparés par une virgule suivie d'un espace. Cette transformation en listes nous a grandement simplifié la tâche lors de la phase 2 du projet, consacrée à la recommandation de films.

Grâce à ces listes, nous avons également pu extraire une liste contenant tous les genres uniques présents dans les données. Cette extraction a permis d'obtenir une vision claire et structurée des différents genres représentés.

Par ailleurs, nous avons choisi de traiter les notes égales à 0 ainsi que celles correspondant à des films n'ayant reçu aucun vote. Plutôt que de les supprimer, nous les avons remplacées par la médiane des notes. Cette décision visait à éviter la perte de données tout en réduisant l'impact des valeurs extrêmes qui auraient pu influencer nos analyses de manière négative.

3.1.4 ANALYSE DESCRIPTIVE ET VISUALISATION DES DONNÉES

Dans cette première partie, nous étions amenés à présenter et visualiser les relations entre les données provenant de TMDB et Netflix à travers différentes questions.

Concernant la représentation de la note moyenne par genre des films, nous avons cherché à construire un Dataframe seulement composé des genres de films en fonction des notes moyennes pour pouvoir plus facilement représenter cela graphiquement.

Aussi, afin de répondre statistiquement de la meilleure des manières aux questions posées dans cette partie, une première étape de transformation des données a été réalisé pour chaque question (conservation de certaines observations seulement, suppression des NaN ou des données égales à 0).

la première question nous avons créé une fonction pour cela, puis nous avons également utilisé un dictionnaire pour apparier les genres selon les notes moyennes. Nous nous sommes interrogés sur s'il fallait remplacer les NaN par la médiane des notes moyennes. Cependant au vu des résultats donnés nous n'avons finalement pas considéré les NaN.

Concernant la deuxième question, une représentation de la répartition des dates de sortie ou d'ajout des films sur une même échelle n'était pas la plus pertinente. Nous avons donc choisi de regrouper l'effectif des films sortis de 1800 jusque 1942. Afin de bien visualiser la répartition des films selon les années nous avons choisi un histogramme.

Enfin, pour expliquer les facteurs expliquant la popularité ou la note d'un film en fonction de nos données, nous avons calculé la corrélation entre les variables quantitatives. Concernant les variables qualitatives nous avons pensé à représenter des scatter plot ou sous forme de camembert.

3.1.5 MOTEUR DE RECHERCHE/RECOMMANDATIONS

Dans cette partie, l'objectif était de concevoir un moteur de recherche permettant de recommander des films à l'utilisateur en fonction de plusieurs caractéristiques. Pour cela, nous avons développé trois fonctions distinctes, chacune étant dédiée à un type de recommandation basé sur des critères différents. Bien que ces fonctions remplissent des objectifs variés, elles reposent sur une logique commune.

Dans un premier temps, nous avons créé des filtres spécifiques que nous avons intégrés à des instructions utilisant la méthode « loc » pour isoler les données pertinentes. Ensuite, les résultats obtenus ont été triés selon les critères demandés ou ceux jugés les plus pertinents (note, popularité, etc.).

Le principal défi résidait dans la gestion des colonnes contenant des listes, qui nécessitaient une logique particulière pour appliquer les filtres correctement. Enfin, chaque fonctionnalité de recommandation demandé est une fonction dynamique, prenant en paramètre les critères de recherche saisis par l'utilisateur. Cette approche garantit une grande flexibilité et une réutilisation facile des fonctions selon les besoins.

3.1.6 CRÉATION D'INTERFACE GRAPHIQUE

L'objectif principal au sein de cette partie consistait à afficher les résultats du moteur de recherche et plus précisément des 3 fonctions distinctes selon des caractéristiques dans une interface graphique.

En premier lieu, nous avons appris à l'aide des liens présents dans le projet comment fonctionnait la librairie Tkinter sur Python. Il a fallu nous exercer à travers des exercices présents sur ces liens afin de comprendre les principales étapes d'affichage d'une interface graphique.

Après avoir compris les fonctionnements de base, nous avons dû mentalement conceptualisé la réalisation complète de l'interface et la dépendance entre les différents boutons qui allaient permettre l'affichage sous forme de data frames des fonctions de la partie 2. Nous avons modifié plusieurs fois la conceptualisation de l'interface selon les difficultés rencontrées.

Par la suite, nous avons créés plusieurs fonctions pour mettre en relations les différents boutons, cette tâche fût complexe car elle demandait l'apprentissage d'affichage simultané de différents types de widgets (checkboxbuttons, entry, listes, label, treeview). Il fallait ensuite définir les widgets les plus adaptés à chaque critère que l'utilisateur pouvait choisir ou entrer et définir des conditions de vérification concernant les entrées saisies par l'utilisateur.

Finalement nous avons utilisé une fonction qui permet d'appeler toutes les fonctions différentes de façon distincte lorsque l'utilisateur appuie sur chacun des 3 boutons principaux définis sur l'interface principale.

3.1.7 MISE AU PROPRE

Une fois tous nos codes écrits et fonctionnels, chacun d'entre nous a rédigé les docstrings dans nos scripts Python ou notebooks afin de rendre le code plus compréhensible. Enfin, nous avons relié toutes nos fonctions en structurant le projet avec des dossiers et des modules. Cela nous permettait d'importer uniquement les fonctions nécessaires, au lieu de regrouper l'ensemble du code dans un seul script Python.

3.2 Les outils utilisés

Au cours de l'étape de préparation des données, nous avons utilisé Excel pour traiter le jeu de données Netflix. Cet outil nous a permis de visualiser les données de manière claire, d'appliquer des filtres facilement, et de les organiser selon nos besoins. Concernant le jeu de données TMDb, nous avons opté pour l'interface Kaggle, qui offre des fonctionnalités similaires pour filtrer et explorer les données de façon intuitive.

Pour les analyses descriptives et la visualisation des données, nous avons utilisé Jupyter Lab, qui s'est révélé particulièrement adapté à ces tâches. Les étapes restantes de notre travail ont été réalisées sur Spyder.

Enfin, pour la communication au sein de l'équipe, nous avons utilisé Microsoft Teams, aussi bien pour nos appels que pour travailler ensemble sur des documents partagés. Le partage de nos codes s'est fait via Wetransfer ou en utilisant une clé USB.

3.3 Les bibliothèques utilisées :

- **Pandas** : bibliothèque offrant des fonctions essentielles pour manipuler et analyser des DataFrames, facilitant le traitement de données structurées
- **Numpy** : bibliothèque dédiée à la manipulation de tableaux multidimensionnels et à l'exécution rapide d'opérations mathématiques sur ces derniers
- **Matplotlib** : outil de visualisation permettant de créer des graphiques et des représentations graphiques simples et personnalisées
- **Seaborn** : extension de Matplotlib, spécialisée dans la création de visualisations statistiques avec des graphiques plus complexes et esthétiques
- **Datetime** : module intégré fournissant des fonctions pour manipuler et formater les dates et heures
- **Re** : module dédié à la gestion des expressions régulières, utile pour effectuer des recherches et manipulations avancées sur des chaînes de caractères
- **Os** : module permettant d'interagir directement avec le système d'exploitation, par exemple pour gérer des fichiers ou accéder à des informations système
- **Tkinter** : bibliothèque standard utilisée pour créer des interfaces graphiques

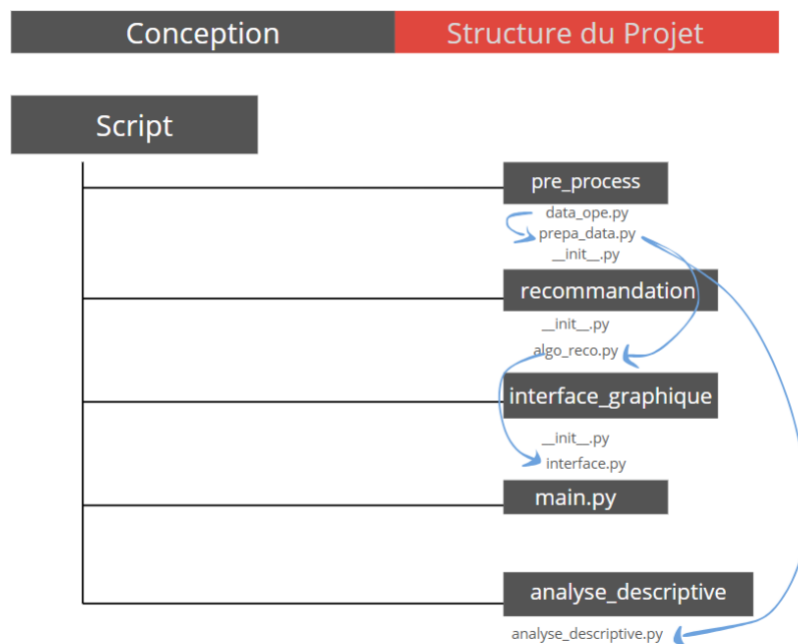
3.4 Répartition des tâches

Pour la répartition des tâches, nous avons décidé de diviser le travail par rapport aux parties qui nous était demandé. Romane s'est occupée de la partie 1, Emma a pris en charge la partie 2 ainsi que la préparation des données, et Jules s'est chargé de la partie 3. Ensuite, nous nous sommes tous soutenus en examinant les parties des autres et en apportant notre aide mutuellement.

En ce qui concerne le rapport, nous avons tous contribué à son écriture et chacun à compléter ses parties assignées. Pour le PowerPoint, nous l'avons réalisé ensemble également.

4. Conception

4.1 Architecture du programme

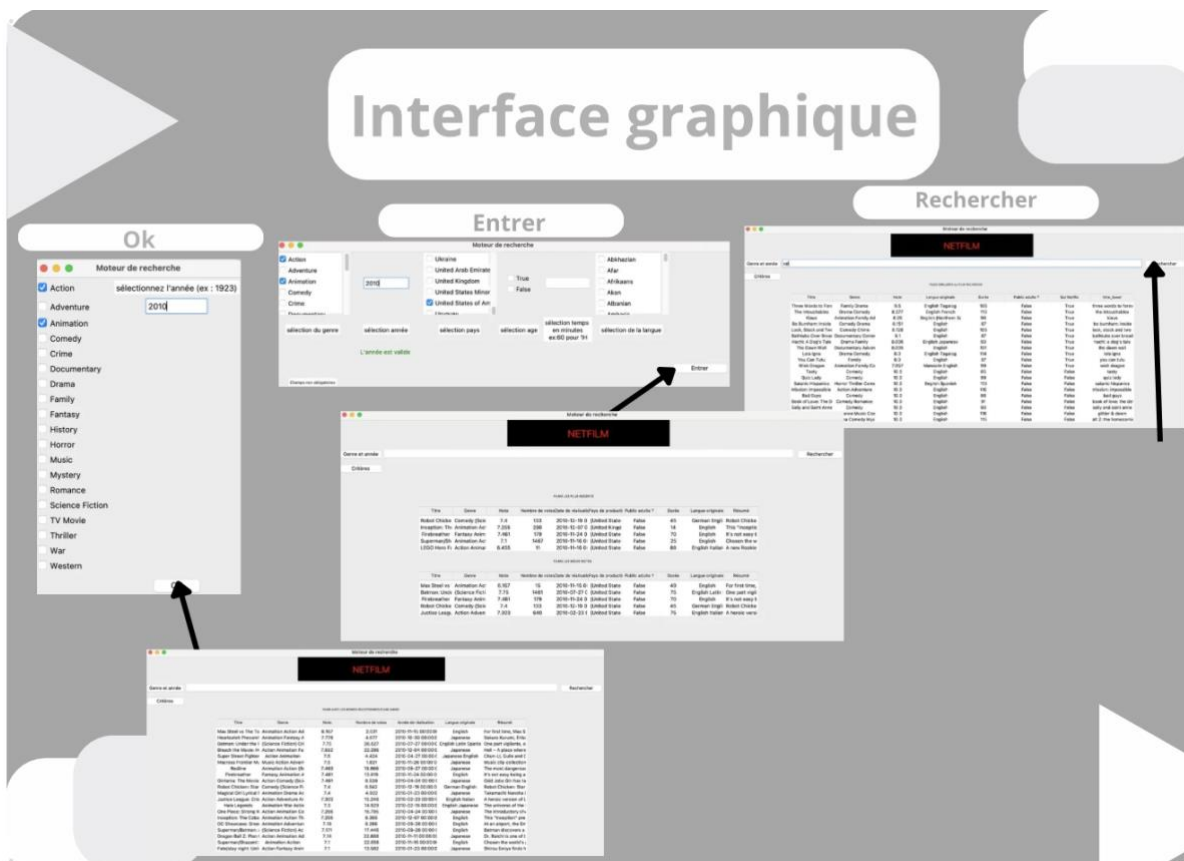
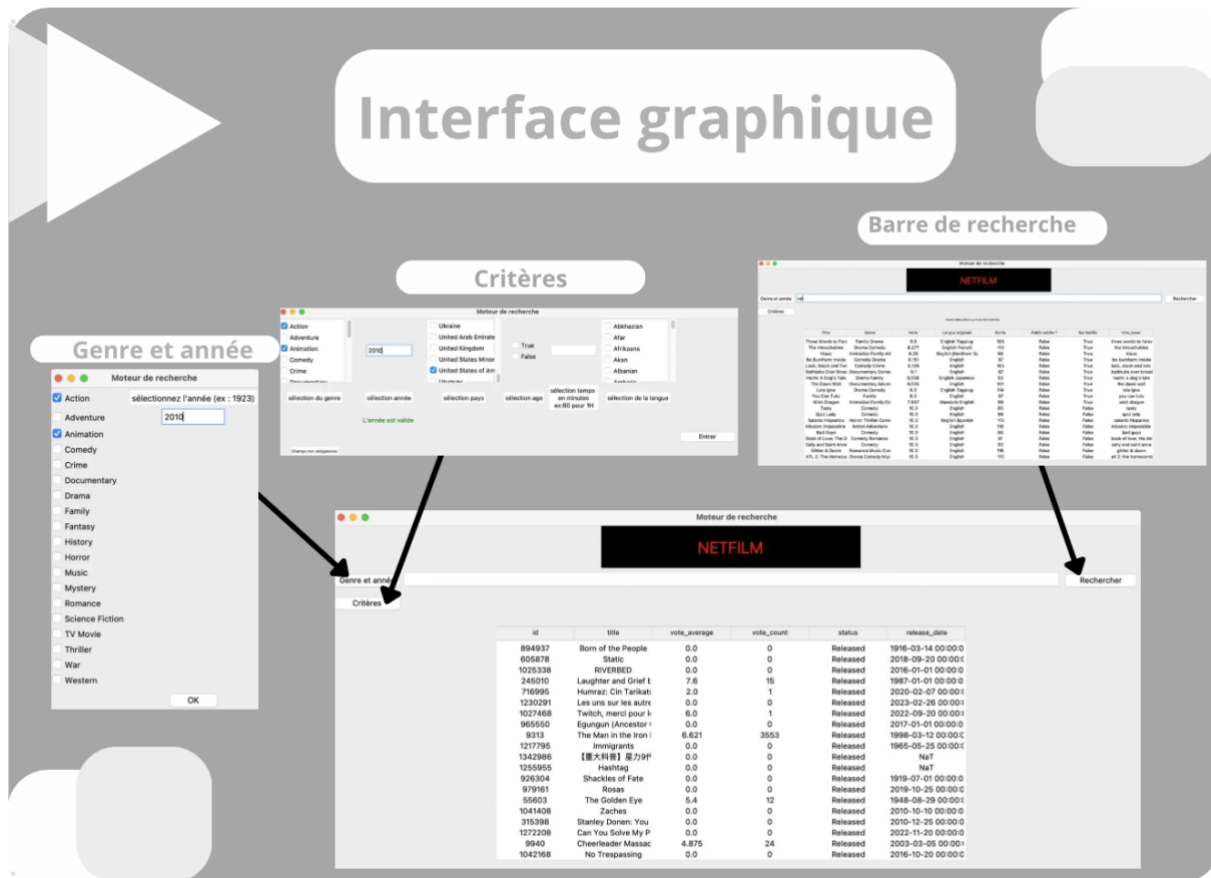


Concernant la structure de notre programme, les différents modules utilisés sont répartis selon nos différentes parties. Tout notre dossier est contenu dans un dossier script qui contient lui-même plusieurs sous-dossiers : `pre_process` (utilisé pour la préparation de données), `recommandation` (utilisée pour définir les fonctions de la partie 2), `interface_graphique` (utilisé pour réaliser le script de l'interface graphique), `main.py` qui est notre script principal et `analyse_descriptive` qui contient le notebook permettant d'afficher les différents graphiques de la partie 1.

Pour l'importation des modules dans les différents scripts, nous avons d'abord importé les fonctions du module « `data_op` » dans le script « `prepa_data` ». Cela a permis d'alléger les scripts en centralisant toutes les fonctions nécessaires dans un fichier dédié, ce qui améliore la lisibilité du code. Par la suite, cela a facilité l'appel de toutes les fonctions requises, tout en nous permettant d'extraire les listes uniques de différentes colonnes. En résumé, cette approche a rendu le code plus clair et mieux organisé. Ensuite, les fonctions du module `prepa_data` ont été utilisées dans d'autres scripts comme `analyse_descriptive` mais également comme interface.

Au sein du document `recommandation` : `algo_reco.py` est importé dans `interface.py` pour assigner les fonctions de la partie 2 dans l'interface graphique tout en prenant en compte les différents critères cochés par l'utilisateur ou non sur les différentes fenêtres. Au final, le `main.py` appelle uniquement par importation le module `interface.py` étant donné l'utilisation déjà établie des fonctions de la partie 2 dans `interface` et le notebook dans la partie 1.

4.2 Interface graphique



5. Implémentation

5.1 Fonctionnalités développées

Tout d'abord au sein de la partie 2, nous avons développés 3 fonctions différentes :

- `filtre_genre_year` : si un utilisateur choisit un ou plusieurs genres et une année, alors nous lui recommandons des films correspondant aux critères suivant leur note et leur popularité, en n'affichant que les noms de films, genre, date de sortie, synopsis, langue, note. Si plusieurs films sont ex-aequo, nous affichons en premier celui qui a reçu le plus de votes
- `filtre_pref` : Filtre les critères de l'utilisateur en laissant la possibilité à l'utilisateur de ne pas sélectionner tous les critères en renvoyant les 5 films les plus récents et les 5 les mieux notés.
- `filtre_title_pref` : l'utilisateur saisit des titres de films appréciés et des titres similaires lui sont renvoyés, basés sur des caractéristiques communes tout en sélectionnant en priorité les films disponibles sur Netflix

Dans la partie 3, nous avons développé plusieurs fonctions qui ont permis l'affichage des différentes fonctions de la partie 2. Nous avons donc créé plusieurs boutons sur la fenêtre principale afin de permettre à l'utilisateur d'isoler les différentes fonctions de manière précise. Ainsi, plusieurs sous fenêtre ont été créées affichant pour chaque fonction les critères demandés (ex : genre et année pour `filtre_genre_year`) puis affichant les data frames obtenus au travers des fonctions des parties 2.

5.2 Exemple de code pertinent

5.2.1 FILTRER LES FILMS SELON DES CRITERES DE PREFERENCE

Cette fonction permet de filtrer un DataFrame en fonction des critères spécifiés dans les paramètres. Par défaut, ces paramètres sont définis à « None ». Il est important de noter que la capture d'écran de la fonction présentée ici ne montre pas la version complète du code. Nous avons sélectionné uniquement les parties les plus pertinentes, par exemple, elle n'inclut pas les triages ou les modifications des titres de colonnes.

```
def filtre_pref(df, genre = None, year = None, pays = None, age = None, time = None, langue = None):  
    # filtre par défaut : l'utilisateur n'a pas la main dessus  
    filtre_nb_vote = df['vote_count'] >= 5  
    filtre_status = df['status'] == 'Released' # pour garder que les films déjà réalisé  
  
    # mise en place des filtres  
    if genre is not None :  
        filtre_genre = df['genres'].apply(lambda x: all(genre_i in x for genre_i in genre))  
    else :  
        filtre_genre = df['genres'].apply(lambda x: True)  
  
    if year is not None :  
        filtre_year = df['release_date'].dt.year == year  
    else :  
        #année de réalisation = à minima la date d'ajd  
        filtre_year = df['release_date'] <= dt.date.today().strftime('%Y-%m-%d')  
  
    if pays is not None :  
        filtre_pays = df['production_countries'].apply(lambda x: all(pays_i in x for pays_i in pays))  
    else :  
        filtre_pays = df['production_countries'].apply(lambda x: True)  
  
    if age is not None :  
        filtre_age = df['adult'] == age  
    else :  
        filtre_age = df['adult'].apply(lambda x: True)  
  
    if time is not None :  
        filtre_time = df['runtime'] == time  
    else :  
        filtre_time = df['runtime'] >= 0  
  
    if langue is not None :  
        filtre_langue = df['spoken_languages'].apply(lambda x: all(langue_i in x for langue_i in langue))  
    else :  
        filtre_langue = df['spoken_languages'].apply(lambda x: True)  
  
    # combine tous les filtres, puis filtres le df seulement si tous les filtres sont True  
    df_filtre = df.loc[filtre_genre &  
        filtre_year &  
        filtre_pays &  
        filtre_age &  
        filtre_time &  
        filtre_langue &  
        filtre_status &  
        filtre_nb_vote  
        ]  
  
    # Affichage du DataFrame filtré  
    return df_sort_recent.head(5), df_sort_note.head(5)
```

Les filtres par défauts :

Nous avons choisi d'appliquer des filtres par défaut sur lesquels l'utilisateur n'a pas de contrôle direct. Tout d'abord, il nous a semblé pertinent de filtrer les films en fonction du nombre de votes. En effet, si un film a moins de 5 votes, il nous paraissait peu judicieux de le recommander, car un faible nombre de votes pourrait ne pas refléter une évaluation fiable. Par exemple, un film avec une seule note de 10 ne serait pas forcément représentatif.

Ensuite, nous avons ajouté un deuxième filtre par défaut, qui consiste à conserver uniquement les films déjà réalisés. Il ne nous paraissait pas cohérent de proposer des films encore en cours de production.

Enfin, pour chaque paramètre, nous vérifions s'il ne prend pas la valeur « None ». Si ce n'est pas le cas, nous appliquons le filtre correspondant. En revanche, si le paramètre est égal à « None », la fonction lambda attribue la valeur « True » à chaque ligne, ce qui permet, lors de l'utilisation de « loc », de ne pas modifier les lignes correspondantes.

Les filtres lorsque le paramètre est complété :

Lors de l'assignation des filtres, nous avons créé différents types de filtres.

Si un paramètre peut contenir une liste, nous utilisons une fonction lambda, qui attribue une valeur « True » ou « False » à chaque ligne en fonction de ce que renvoie la fonction « all ». Cette fonction vérifie si chaque genre (représenté par « genre_i ») de la liste « genre » est présent dans « x » (l'élément de la colonne « genres » du DataFrame). Ici, « x » est une liste de genres. La fonction « all » renvoie « True » ou « False » : si au moins un genre de la liste fournie par l'utilisateur n'est pas présent dans les genres de la ligne du DataFrame, la fonction renvoie « False » et la ligne sera exclue du résultat. En revanche, si tous les genres sont présents, la fonction renvoie « True », et la ligne sera conservée. La méthode « .apply() » permet d'appliquer cette logique à chaque ligne de la colonne spécifiée, ici la colonne « genres ».

Enfin, si un paramètre contient uniquement une seule valeur, nous utilisons un modèle de fonction similaire à une fonction « loc » basique. Dans ce cas, nous sélectionnons uniquement les lignes qui répondent à notre condition. Par exemple, pour l'âge, si l'utilisateur entre « False », la ligne de code va vérifier, pour chaque ligne, si la colonne « age » contient « False » ou non. Elle renverra « True » si c'est le cas, et « False » sinon.

Établissement de tous les filtres

Enfin, tous les filtres sont regroupés dans notre fonction « loc », qui ne sélectionnera que les lignes où toutes les conditions sont remplies (où chaque condition renvoie « True »). Ensuite, nous retournons uniquement les 5 premiers films du DataFrame, sachant que ce dernier a été trié selon certaines conditions (qui ne sont pas présentées ici, mais font partie du code complet de la fonction).

5.2.2. EXEMPLE : EXPLICATION DANS L'INTERFACE DE L'AFFICHAGE DU BOUTON2

Pour éviter un affichage trop lourd, nous avons choisi de ne pas mettre de capture d'écran de ce code

Tout d'abord nous créons une sous fenêtre grâce à `TopLevel` puis différents `checkboxbuttons` au sein de `scrolltext` afin de pouvoir sélectionner puis scroller à travers les différents critères. Par critère, nous définissons un `frame` afin de permettre l'affichage sur la même sous fenêtre des différents critères. Pour les genres, langues et pays le code repris est presque identique. Tout d'abord nous initialisons une liste vide et réalisons une boucle qui va dans la liste crée correspondant à celle des genres, pays et langues. Nous créons une variable `var` qui prendra 1 si le `checkboxbutton` d'une ligne est cochée et 0 sinon.

Nous relierons donc chaque checkbox à une variable à l'aide de `variable=var` et chaque genre, pays, langue s'affiche grâce à `text=str(index)` puis nous stockons toutes ces variables dans la liste initialisée précédemment. Un Label est créé pour chaque critère demandé pouvant indiquer à l'utilisateur les critères à cocher. On place ensuite grâce à grid les différents widgets sur la sous fenêtre.

Concernant le temps, l'année, il était plus pertinent de rentrer des entry étant donné le nombre trop important de variables différentes. On a donc jugé plus pertinent de mettre des critères de sélection dans la fonction traitement bouton 2 afin de vérifier si les données étaient ou non valides. Concernant la variable age, qui est une booléenne, nous avons seulement créé 2 boutons pouvant renvoyer True or False.

Un bouton Ok a ensuite été créé afin d'afficher la fonction de traitement permettant ainsi de voir quels critères ont été sélectionnés puis d'afficher le data frame.

5.2.3 EXEMPLE : EXPLICATION DANS L'INTERFACE DU TRAITEMENT DU BOUTON2

Pour éviter un affichage trop lourd, nous avons choisi de ne pas mettre de capture d'écran de ce code

On commence par définir en global les différentes listes et variables qui seront réutilisées en tant que paramètres dans la fonction `reco.filtre_pref`. Pour cela il faut savoir sous quelle forme récupérer les différentes listes et variables. Pour les listes de str telles que les langues, les pays, les genres il suffit d'initialiser une liste vide. On crée ensuite une boucle qui va jusqu'au dernier index des listes dans lesquelles les variables précédentes des checkboxes ont été stockées. Comme l'index correspond à celui des listes créées en partie 2 ex pour les listes genres, pays, langues cela ne crée pas de problème d'indexation. Si une variable a été cochée alors elle prendra 1 et sera donc mise grâce à `append` sous forme str dans la nouvelle liste de langues (par ex) qui pourra être utilisée comme paramètre dans la fonction `reco.filtre_pref`. Si aucun bouton n'a été coché alors la variable prendra la valeur None. Concernant l'année, le temps une vérification a lieu pour vérifier si la variable est sous forme de nombre et est inclus dans la liste correspondante à celle créée en partie 2. Si c'est le cas, la variable devient int et pourra ainsi être utilisée en paramètre de la fonction, sinon elle prendra None

5.2.4. EXEMPLE : EXPLICATION D'AFFICHAGE DE 2 DATA FRAMES SUR L'INTERFACE

On utilise d'abord la fonction `reco.filtre_pref` avec en paramètre `df`, les genres sélectionnés, l'année, la liste des pays, la variable age, la valeur temps, la liste des langues sélectionnés et ressort 2 data frames. Par la suite, on supprime le précédent frame sur lequel était affiché l'ancien data frame avec `for widget in frm7.winfochildren():`

```
    widget.destroy()
```

On vérifie si le data frame est vide ou non et si c'est le cas, la fonction renvoie un texte affiché « Aucun résultat trouvé ». Si le dataframe n'est pas vide la sous fenêtre se ferme afin de concentrer l'intégralité de l'attention de l'utilisateur sur le data frame de la partie 2. On utilise donc un treeview relié au data frame afin de réaliser l'affichage.

```

#AFFICHAGE DES 2 DATA FRAMES
#Utilisation de la fonction reco.filtre_pref
df_sort1, df_sort2=reco.filtre_pref(df,liste_genre_selectionnees, valeurannee2,
                                   liste_pays_selectionnees, valeurage, valeurtemps, liste_langues_selectionnees)

#Permet de supprimer les anciens data frames affichés sur le widget frm7 et d'en créer un nouveau
global frm7
for widget in frm7.wininfo_children():
    widget.destroy()
frm7=tk.Frame(fenetre)
frm7.grid(column=1, row=3)

#Creation d'une liste des colonnes
l1 = list(df_sort1.columns)

#Si le data frame en sortie est vide, afficher Aucun résultat trouvé
if df_sort1.empty:
    tk.Label(frm7, text="Aucun résultat trouvé", fg="black").grid(row=1,column=1, padx=10)
    return

#Convertir le data frame en liste de liste
root.destroy()
r_set = df_sort1.to_numpy().tolist()

#Utilisation d'un Treeview pour afficher le data frame
trv = ttk.Treeview(frm7, selectmode="browse")
trv.grid(row=1, column=1, columnspan=5, padx=10, pady=20) #

#création d'un Label affichant le but de la fonction
text1=ttk.Label(frm7, text="FILMS LES PLUS RECENTS", font=("Time", 8), width=31, foreground="black", background="white")
text1.grid(row=0, column=1,columnspan=5, sticky="")

#Définition du nombre de lignes, colonnes et variables
trv["height"] = 5
trv["show"] = "headings"
trv["columns"] = l1

#création d'une boucle pour définir les colonnes et noms de colonnes de tout le df
for col in l1:
    trv.column(col, width=90, anchor="c")
    trv.heading(col, text=col)

#création d'une boucle qui parcourt dans la liste de liste les lignes du df
for row in r_set:
    trv.insert("", "end", values=row)

```

Au sein de cette fonction 2 data frames seront affichés ce qui en fait une spécificité. Sans l’affichage de ces 2 data frames il aurait été possible d’utiliser une fonction afin d’afficher tous les data frames de façon globale entre les différentes parties. Cependant ici, il n’a pas été possible de fermer frm7 sous peine de n’avoir qu’un seul data frame.

Nous créons donc une liste de liste avec les différentes colonnes des data frames puis nous définissons le nombre de ligne (ici 5 et 5) et le nombre de colonnes avec le contenu des variables. Par la suite, nous créons 2 boucles allant jusqu’au nombre de lignes du data frame et jusqu’au nombre de colonnes afin de remplir sur 5 lignes les data frames avec les films les plus récents et les mieux notés sortis de la fonction reco.filtre_pref.

Finalement l’instruction barre.delete(0, tk.END) permet de supprimer l’entry barre donc si l’utilisateur a rentré un nom de film dans la barre de recherche, il s’effacera.

6. Tests et validations

Dans un premier temps, pour la partie recommandation, nous avons effectué les tests en utilisant Jupyter Lab, en attendant d’avoir l’interface graphique. Cela nous a permis de visualiser plus facilement les DataFrames que sur Spyder. Dans ce cas, nous importons le module destiné à préparer les données, puis nous intégrons notre fonction et utilisons un exemple (ces exemples sont présents dans nos docstrings ainsi qu’à la fin de notre module). Bien que nous ayons utilisé la structure « if

`__name__ == '__main__':` », celle-ci ne pouvait pas être pleinement exploitée, car nous ne réalisons pas les tests directement sur Spyder.

Afin de réaliser l'interface graphique, les tests réalisées étaient plus simples car nous avons isolé de manière distincte les boutons qui permettaient chacun l'affichage d'une fonction de la partie 2. Ainsi lorsqu'une fonctionnalité attendue ne marchait il était possible de repérer dans quelle fonction il était assigné. Si un widget ne s'affichait pas correctement ou n'affichait rien, il suffisait d'aller dans la fonction d'affichage du bouton concerné. Si le data frame ne s'affichait pas, le problème provenait des fonctions de traitement de chaque bouton ou de la fonction qui créait la première fenêtre. Comme vérification des critères sélectionnés par l'utilisateur nous avons utilisé des `print` afin de vérifier si les critères attendus avaient le même format que les paramètres utilisés dans les diverses fonctions. Si ce n'était pas le cas nous modifiions le type de données attendu.

De plus, nous avons utilisé des noms de film, des critères de manière habituelle afin d'observer si les résultats des data frames étaient cohérents

7. Perspectives

7.1 Les améliorations possibles

Dans un premier temps, nous pensons qu'il aurait été possible d'améliorer davantage le taux de concaténation. Une meilleure répartition du travail aurait également permis à chacun de participer à toutes les étapes du projet, favorisant ainsi une vision globale et collaborative.

De plus, nous avons constaté qu'il aurait été judicieux d'appliquer nos modifications sur de nouvelles colonnes créées, comme par exemple une colonne dédiée aux genres. Cela aurait également permis un affichage plus esthétique, notamment en évitant les listes dans notre interface graphique.

Par ailleurs, nous aurions aimé écrire le script de l'interface en plusieurs fichiers distincts. Bien que nous ayons essayé de le faire, nous n'avons pas obtenu les résultats attendus (car les variables entre les différents scripts n'étaient pas reconnaissables et dépendaient les uns des autres). Nous avons donc opté pour un script unique. Nous sommes conscients que cela a généré un grand nombre de lignes de code, rendant le script potentiellement difficile à lire. C'est pourquoi nous avons apporté un soin particulier à nos docstrings, afin de le rendre aussi clair que possible.

De plus, nous pensons que nous aurions pu utiliser des noms de variables plus explicites et sans fautes. En effet par convention sur Python, les noms des boutons sont associés à des noms anglophones ex : `button`. Cependant dans notre code, nous avons mélangé l'écriture française et anglaise rendant moins lisible l'utilisation de variable avec « `button` », « `bouton` » ou « `boutton` ».

Il aurait été également plus pertinent de créer une fonction pour afficher chaque data frame et réduire le ligne de codes. Cependant, malgré plusieurs essais au sein de la 2^{ème} fonction, l'utilisation répétée de l'affichage de ces 2 data frames sur une seule interface ne permettait pas d'afficher les 2 de manière simultanée.

Nous aurions pu réutiliser les fonctions créées dans la partie 2 pour effectuer nos analyses graphiques et descriptives des données. En recréant certaines fonctions, nous avons introduit des redondances que nous aurions pu éviter. Par exemple, la fonction « `reco_filtre_pref` » aurait pu être adaptée pour calculer les moyennes de chaque genre, puis générer un dictionnaire associant chaque genre à sa moyenne.

Enfin, bien que ce soit un détail, nous aurions aimé améliorer et rendre notre interface plus esthétique en créant d'autres boutons et fonctions possibles lorsque l'utilisateur ne clique pas forcément à l'aide d'un clic droit sur les différents boutons.

7.2 Apprentissages et difficultés rencontrées au cours du projet

Au cours de ce projet, nous avons appris à travailler en groupe, à répartir les tâches efficacement et, surtout, à maintenir une communication fluide, ce qui est essentiel étant donné que nos codes interagissent tous entre eux.

Cependant, nous avons tous rencontré une difficulté liée à la gestion de la RAM de nos ordinateurs. En effet, chacun de nous a fait face à des problèmes de saturation de la RAM lors de l'exécution des scripts Python. Pour y remédier, nous avons ouvert le moniteur d'activité et supprimé tous les processus liés à Python afin d'alléger la mémoire.

ROMANE :

- Les apprentissages : Au cours de ce projet, j'ai approfondi mes compétences en Python, notamment en ce qui concerne l'utilisation des fonctions, des listes et des DataFrames. J'ai également amélioré ma capacité à structurer et moduler mon code de manière efficace. Par ailleurs, je me suis familiarisé avec l'utilisation des bibliothèques Matplotlib et Seaborn, des outils que je considère précieux pour mon futur métier. Globalement, cette expérience m'a permis de gagner en fluidité dans la programmation et de développer une réflexion plus rigoureuse sur la logique nécessaire pour coder efficacement en Python. J'ai également beaucoup appris avec l'utilisation de fonctions `loc` et `iloc` ainsi que les fonctions `lambda` nécessaires pour nettoyer, transformer les données afin de pouvoir les visualiser par la suite tout en essayant que ces données soient statistiquement pertinentes.
- Les difficultés : Concernant les difficultés rencontrées, j'ai souvent consacré trop de temps à des approches trop complexes dès le début, au lieu de commencer par des graphiques ou des analyses simples. De plus, une part importante de mon temps a été dédiée à la transformation des données pour les rendre exploitables dans mes graphiques. Enfin, j'ai également été confronté à des problèmes liés à la lenteur de mon ordinateur, ce qui a ralenti mon travail.

EMMA :

- Les apprentissages : Au cours de ce projet, j'ai appris à utiliser de nouvelles bibliothèques, telles que « `re` », ce qui me sera utile, car il est parfois nécessaire d'uniformiser les données avant de pouvoir les analyser. De plus, j'ai désormais une bonne maîtrise de la fonction « `loc` », qui permet d'extraire un nouveau DataFrame en fonction de critères spécifiques. Enfin, j'ai réellement appris à utiliser et comprendre les fonctions « `lambda` », ce qui m'a permis d'alléger mon code et de le rendre plus lisible.
- Les difficultés : J'ai rencontré de nombreuses difficultés, notamment lors de la préparation des données. Je n'arrivais pas à trouver la méthode de concaténation la plus optimale, jusqu'à ce que je découvre la bibliothèque « `re` », ce qui a grandement facilité ce processus par la suite. En ce qui

concerne le filtrage sur une liste, cela a été l'un des aspects les plus difficiles au début, car je ne savais pas comment le mettre en place. Cependant, après avoir étudié des exemples d'utilisation de la fonction « lambda », j'ai pu la comprendre et l'adapter à mes besoins. Enfin, un autre défi a été d'optimiser mon code pour le rendre plus concis.

Au final, mes difficultés reflètent mes apprentissages, car c'est en affrontant ces challenges que j'ai réellement appris et assimilé de nouvelles connaissances.

JULES :

- Les apprentissages : En travaillant sur ce projet, j'ai acquis de nouvelles compétences à travers de multiples étapes. Tout d'abord j'ai appris à utiliser une bibliothèque « Tkinter » que je ne maîtrisais pas, cela m'a rassuré sur ma capacité de compréhension qui me sera utile en entreprise. En effet, je devrai apprendre sans cesse de nouveaux aspects et réussir à les mettre en œuvre. De plus, comme j'avais comme mission la réalisation de l'interface graphique, il fallait que j'arrive à dépasser certains blocages afin de rendre l'interface la plus optimale et esthétique possible. J'estimais aussi que cela représentait une part capitale du projet, qui montre directement le résultat fourni dans l'ensemble du dossier car les parties sont dépendantes entre elles. Je pense avoir fortement amélioré ma capacité à trouver des erreurs et à structurer un code étant donné la complexité d'un grand nombre de lignes.
- Les difficultés : Au sein de la partie 3, j'ai été confronté à plusieurs difficultés. En premier lieu, l'acquisition de connaissances d'une nouvelle librairie prit du temps pour ma part. J'ai également eu du mal à structurer de façon conquise l'ensemble de mon code ainsi que la conception graphique de mon interface. Pour cela Emma et Romane m'ont aidé en n'hésitant pas à me dire les améliorations possibles. La dernière difficulté la plus importante pour ma part consistait à trouver d'où provenait les erreurs à chaque nouvelle étape. Cela m'a forcé à prendre des mesures de vérification de manière isolée pour les différents boutons. Par exemple, au départ je ne savais pas qu'il était possible de créer plusieurs sous-fenêtres (alors que j'avais créé plusieurs fenêtres). Cependant la relation entre les différentes fenêtres entraînait une incohérence sur la librairie.

8. Annexes

8.1 Les instructions pour lancer notre programme

Pour démarrer le programme, ouvrez Spyder et chargez le fichier « main » situé dans le répertoire : Sujet 1_BOUGEARD_CRISMAT-LE-DIRACH_KNIERIEMEN/script. Ensuite, sélectionnez le répertoire de travail correspondant, c'est-à-dire le dossier où se trouve le script. Assurez-vous d'avoir le chemin relatif correct du dossier « Sujet 1_BOUGEARD_CRISMAT-LE-DIRACH_KNIERIEMEN/script ». Il suffit ensuite de lancer le script. Attention à ne pas mettre l'affichage de l'écran en mode sombre, sinon les couleurs de l'interface ne seront pas adaptées.

8.2 Bibliographie

- [Fonction qui permet de récupérer uniquement l'année d'un format Datetime : dt.year](#)
- [Fonction qui permet de supprimer les doublons dans un DataFrame : drop_duplicates](#)
- [Explication sur les différents types de jointures disponibles avec la fonction merge](#)
- [Fonction permettant de vérifier des éléments et qui renvoie True ou False : all\(\) \(fonctionne comme any\(\)\)](#)
- [Fonction permettant de trier un DataFrame : sort_values\(\)](#)
- [Explication de la bibliothèque re dédiée aux expressions régulières](#)
- [Guide complet sur l'utilisation des expressions régulières](#)
- [Fonction pour récupérer la date d'aujourd'hui et gérer le format des dates : dt.date.today\(\)](#)
- [Explication et compréhension de la fonction lambda ainsi que de son utilisation avec apply](#)

- [Explication sur comment utiliser la fonction display](#)
- [Itération des lignes dans un Dataframe](#)
- [Fonction pour maximiser les performances](#)
- [Merge](#)
- [Concaténation df](#)
- [Nettoyage de données](#)
- [Sélectionner et indexer les données](#)
- [Fonctions Loc\(\) et iloc\(\)](#)
- [Fonction .where](#)
- [Fonction Loc](#)
- [Fonction Loc 2](#)
- [Convertir séries en listes](#)
- [Radar Chart](#)
- [Fonction corr\(\)](#)
- [Fonction corr\(\) 2](#)

- [Apprentissage de la librairie Tkinter au départ](#)
- [Tutoriel interface graphique tkinter](#)
- [Apprentissage de la librairie Tkinter](#)
- [Barre de recherche qui se remplit automatiquement](#)
- [Fonctionnement du widget entry](#)
- [Boîtes de dialogue tkinter](#)
- [Affichage du scrollbar](#)
- [Création de checkbuttons sous forme de liste](#)
- [Affichage data frame sur Tkinter](#)
- [Réaliser faire une checkbox avec une liste](#)
- [Suppression d'un widget Treeview](#)
- [Effacer un cadre dans Tkinter](#)
- [Modifier la taille d'un texte dans un Frame](#)
- [Centrer un widget avec grid](#)
- [Obtenir l'input d'une checkbox](#)
- [Mettre les checkbuttons sous forme de liste](#)
- [Utiliser la fonction bind sur un bouton](#)
- [Checkbuttons Tkinter](#)
- [Avoir l'état de plusieurs checkbuttons](#)
- [Utilisation de sous fenêtres](#)
- [Widget scrollbar](#)

- Listbox widget
- Heatmap Seaborn
- Couleur Heatmap
- Fonction pour diviser des données continues en intervalles défini : pd.cut()
- Référence Pandas pd.cut()